

Flush Equivalence Implies Interleaved Equivalence for Stateless PIFO-Tree Schedulers

Codex and Claude

1 Definitions

Definition 1 (PIFOs and finite topologies). *A PIFO is a priority queue whose entries have a natural-number rank and an arrival index. A pop removes an entry of lowest rank, with ties broken in favor of the earliest arrival. A finite PIFO-tree topology is a finite rooted tree with a PIFO at every node. A leaf PIFO holds packets, while an internal-node PIFO holds indices of its children.*

Definition 2 (Ranked path assignments and stateless schedulers). *Let A be a finite set of packet types. A ranked path assignment on a finite topology assigns to each $a \in A$ a root-to-leaf path and a natural-number rank at every node of that path. At an internal node the corresponding entry value is the index of the next child, and at the leaf it is the packet type a . A stateless PIFO-tree scheduler S consists of a finite topology and a fixed ranked path assignment: every push of type a uses that same annotated path, independently of the current state and arrival number. Schedulers being compared may have different finite topologies.*

Definition 3 (States and push semantics). *A state assigns a finite PIFO to every node and is initially empty. A push of type a receives a fresh global arrival index and walks the assigned path for a , enqueueing at each visited node the prescribed value and rank together with that arrival index. Thus FIFO tie-breaking is unambiguous, and the entries created by one push may be ghost-tagged by that source push. The observable packet value is its type.*

Definition 4 (Pop semantics and atomic failure). *A pop removes the selected entry from the root. At an internal node its value chooses a child, and the pop recurses there; at a leaf its value is the emitted packet type. The operation is atomic: if a required queue is empty, the entire state is left unchanged and the observable result is **none**; otherwise all removals commit and the result is **some**(a) for the emitted type a .*

Definition 5 (Operation words and runs). *An operation word is a finite word over operations $\text{push}(a)$, for $a \in A$, and pop . The run of S on an operation word starts from the empty state, performs its operations in order, and records, in order, the **none** or **some**(a) result of every pop.*

Definition 6 (Flush words and flush equivalence). *For $w = a_1 \cdots a_n \in A^*$, its flush word is*

$$\text{push}(a_1) \cdots \text{push}(a_n) \text{pop}^n.$$

Its emitted type word is denoted $F_S(w)$. Two schedulers S, T are flush-equivalent if $F_S(w) = F_T(w)$ for every $w \in A^$.*

Definition 7 (Interleaved equivalence and notation). *Schedulers S, T are interleaved-equivalent, written $S \simeq_{\text{inter}} T$, if their runs have identical recorded outputs on every operation word. For $B \subseteq A$, let $w|_B$ be word projection. For a total preorder r , let $\text{sort}_r(w)$ be the stable sort by $(r(a), \text{arrival})$. For a partition P of A , let $\pi_P(w)$ replace every type by its P -block. Restriction to B retains only the assigned paths of types in B and prunes inactive parts of the topology.*

2 Operational facts

Lemma 8 (Balance and timestamp compression). *At every reachable state and every internal-node child c ,*

$$\#\{\text{references to } c\} = \#\{\text{packets below } c\}. \quad (1)$$

*A pop fails exactly when the total packet count is zero, so its **none** or **some** status depends only on the operation word. Moreover, replacing all arrival stamps participating in a projected child run by $1, 2, \dots$ in their original order preserves that run.*

Proof. A valid push increments both sides of (1), and a successful pop decrements both. Hence a selected reference always points to a nonempty subtree, so the partial-failure branch of atomic pop is unreachable. A pop therefore fails exactly when the total packet count is zero, and the **none** or **some** pattern depends only on the operation word.

Only comparisons of arrival stamps matter. Replacing any increasing subsequence of global timestamps that constitutes a projected child run by $1, 2, \dots$ preserves every queue decision in that run. Thus embedded child runs with timestamp gaps equal their standalone runs after timestamp compression.

We may ghost-tag an entry by its source push. This tag is not observable and is not stored in the entry value; erasing it commutes with every transition because queue ordering uses only rank and arrival. $\square$

Lemma 9 (Two-type lemma). *For distinct x, y , every operation word over $\{x, y\}$ behaves as one stable PIFO with an effective comparison*

$$e(x, y) \in \{x < y, x = y, y < x\}.$$

The two binary flushes recover this comparison exactly.

Proof. Follow the paths of x, y to the first internal node where their child indices differ, or to their common leaf if they never differ. Before that decisive node, every reference has the same child value, so those nodes merely forward one anonymous service per successful pop.

At the decisive node, if it is internal, its selected child identifies x or y , and that child's restricted subtree is monochromatic; if it is a leaf, the queue directly emits x or y . The effective comparison is recovered by

$e(x, y)$	$F(xy)$	$F(yx)$
$x < y$	xy	xy
$x = y$	xy	yx
$y < x$	yx	yx

(2)

This includes FIFO ties. $\square$

Lemma 10 (Colored-PIFO lemma). *Let p, q be two total preorders on types, colored by γ , and suppose*

$$\gamma(x) \neq \gamma(y) \implies \text{cmp}_p(x, y) = \text{cmp}_q(x, y), \quad (3)$$

including equality. Then queues ordered by p, q , receiving identical pushes and pops, emit identical color traces.

Proof. Partition each color into cross-profile classes:

$$x \sim y \iff \gamma(x) = \gamma(y) \wedge \forall z \text{ of another color, } \text{cmp}_p(x, z) = \text{cmp}_p(y, z).$$

These classes are common to p, q . Distinct classes have a common quotient preorder: differently colored classes are ordered identically by (3); if same-colored classes have different profiles, a witnessing outside type places one class strictly before the other in both preorders.

If a class ties another class, the other has a different color, and transitivity forces every member of the first class to tie every member of the other class under both p, q .

Maintain equal pending counts in every profile class, and identical pending occurrence sets in every class tied with another class. A pop chooses the same least quotient class. If several classes tie, FIFO chooses the same globally earliest occurrence. If the least class is an isolated monochromatic class, the queues may remove different types inside it, but emit the same color and leave equal counts. The invariant survives later pushes and pops. Empty pops agree as well. $\square$

3 Flush decompositions

Definition 11 (Decomposition). *For a word transformer $F: A^* \rightarrow A^*$, a proper partition P of A and a total preorder r form a decomposition (P, r) of F when, for every $B \in P$ and $w \in A^*$,*

$$F(w)|_B = F(w|_B) \quad \text{and} \quad \pi_P(F(w)) = \pi_P(\text{sort}_r(w)).$$

Lemma 12 (Top decomposition). *Every stateless scheduler on A with $|A| > 1$ is online-equivalent to*

$$M = (P, r, (M_B)_{B \in P}), \tag{4}$$

where P is a proper partition of A into active root children, r is the root preorder, and M_B is the restricted child scheduler. Its flush transformer satisfies

$$F_M(w)|_B = F_{M_B}(w|_B) = F_M(w|_B) \tag{5}$$

and

$$\pi_P(F_M(w)) = \pi_P(\text{sort}_r(w)). \tag{6}$$

Thus (P, r) is a decomposition of F_M .

Proof. Follow the unique child used by all types until reaching either the first node with at least two active children or a leaf. Nodes skipped this way are online-inert: every reference has the same child value. At a terminal leaf, replace it by a root carrying the old leaf ranks and one FIFO singleton child per type. The expanded root mirrors the old leaf queue occurrence-for-occurrence. This gives (4).

Equation (5) holds because child B receives all of $w|_B$ before any service and is called exactly $|w|_B$ times. Equation (6) holds because the root drains its references in stable r -order. $\square$

4 The meet lemma

Lemma 13 (Meet lemma). *If (P, r) and (Q, s) decompose the same transformer F , then for some total preorder t ,*

$$\boxed{(P, r) \text{ and } (Q, s) \text{ decompose } F \implies (P \wedge Q, t) \text{ decomposes } F.} \quad (7)$$

Proof. Let

$$R = P \wedge Q = \{B \cap C \neq \emptyset : B \in P, C \in Q\}.$$

For a cell $D = B \cap C$,

$$\begin{aligned} F(w)|_D &= (F(w)|_B)|_C \\ &= F(w|_B)|_C \\ &= F((w|_B)|_C) \\ &= F(w|_D). \end{aligned} \quad (8)$$

Thus every meet cell is autonomous.

For types in distinct R -cells, prescribe the binary effective comparison $e(x, y)$ from (2). If P separates x, y , then their pair restriction is selected at the P -root, so

$$e(x, y) = \text{cmp}_r(x, y). \quad (9)$$

Likewise, if Q separates them,

$$e(x, y) = \text{cmp}_s(x, y). \quad (10)$$

Hence the prescriptions agree whenever both partitions separate the pair.

Take one type from each of three distinct cells, viewed as points in the $P \times Q$ incidence grid. If their P -coordinates are distinct, all comparisons come from r . If their Q -coordinates are distinct, all come from s . Otherwise the three points form an L. After naming its common corner x , the restricted transformer has both decompositions

$$\{x, y\} \mid \{z\}, \quad \{x, z\} \mid \{y\}. \quad (11)$$

We prove that e on this triple is a total preorder. Otherwise its non-strict relation is not transitive, so there are distinct a, b, c with

$$a \leq_e b, \quad b \leq_e c, \quad c <_e a.$$

Push them in the single arrival order $\sigma = abc$. This orients any tie on a, b or b, c forward, producing a stable precedence cycle. Rotate its names around the common corner so that

$$x \prec_\sigma y \prec_\sigma z \prec_\sigma x. \quad (12)$$

Crucially, σ is retained; it need not be the word xyz .

For the first decomposition in (11), the outer occurrence order is y, z, x , giving slots

$$\{x, y\}, z, \{x, y\}.$$

The inner pair outputs x, y , so the flush output is x, z, y . For the second decomposition, the outer order is x, y, z , giving slots

$$\{x, z\}, y, \{x, z\}.$$

Its inner pair outputs z, x , so the result is z, y, x . This contradicts the fact that both are decompositions of the same F . Thus every three-cell restriction is a total preorder.

Build a directed constraint graph on types: $x <_e y$ contributes a marked edge $x \rightarrow y$, while $x =_e y$ contributes unmarked edges both ways. These comparisons extend to a total preorder exactly when there is no directed cycle containing a marked edge. Indeed, if no such cycle exists, no marked edge lies inside a strongly connected component. Collapse strongly connected components, linearly extend the resulting directed acyclic graph, and assign successive natural ranks; ties lie inside components and strict comparisons go between them.

Suppose a shortest marked closed walk existed. It may be taken simple. If vertices two steps apart belonged to distinct cells, the total preorder on those three cells would provide a shortcut, marked whenever either replaced edge was marked. This would give a shorter marked closed walk.

Therefore every second vertex lies in the same cell. An odd cycle is then impossible, and an even cycle alternates between two cells. But all comparisons between two fixed cells come from one genuine preorder: from r if their P -blocks differ; otherwise from s , since their Q -blocks differ. No genuine preorder contains such a cycle. Hence the required total preorder t exists and is realizable by natural ranks. Moreover,

$$t = r \quad \text{across distinct } P\text{-blocks}, \quad t = s \quad \text{across distinct } Q\text{-blocks}. \quad (13)$$

By the colored-PIFO lemma, with colors P ,

$$\pi_P(F(w)) = \pi_P(\text{sort}_r(w)) = \pi_P(\text{sort}_t(w)). \quad (14)$$

Fix $B \in P$. Inside B , its R -cells are its intersections with Q -blocks. Therefore

$$\begin{aligned} \pi_Q(F(w)|_B) &= \pi_Q(F(w|_B)) \\ &= \pi_Q(\text{sort}_s(w|_B)) \\ &= \pi_Q(\text{sort}_t(w|_B)) \\ &= \pi_Q(\text{sort}_t(w)|_B). \end{aligned} \quad (15)$$

The middle equality is colored congruence using (13). The last is stable-sort projection: deleting occurrences outside B changes no retained pairwise order; compressing timestamps is strictly increasing.

The global P -word together with every per- B Q -subword uniquely determines the R -word. Scanning the P -word, at each B -position take the next Q -symbol for that B . Thus

$$\pi_R(F(w)) = \pi_R(\text{sort}_t(w)). \quad (16)$$

Equations (8) and (16) prove the meet lemma. Equivalently, F can be realized by a t -root routing to the R -cells, with child transformer $F|_D$ in cell D . A word is uniquely reconstructed from its cell skeleton and its subsequence in every cell. $\square$

5 The theorem

Theorem 14. *For stateless PIFO-tree schedulers over arbitrary finite topologies and a finite packet-type set, flush-equivalence implies interleaved-equivalence.*

Proof. We prove the theorem simultaneously for all finite alphabets A .

For $|A| = 0$, every operation is a failed pop. For $|A| = 1$, each successful pop emits the sole type, and success is determined by the packet count.

Let $|A| > 1$, and normalize two flush-equivalent schedulers S, T as above. Let their proper top decompositions be

$$(P, r, (S_B)), \quad (Q, s, (T_C)),$$

with common flush transformer F .

Apply the meet lemma, obtaining $R = P \wedge Q$ and t . For each $D \in R$, choose a valid scheduler U_D realizing $F|_D$, for example the restriction $S|_D$. Let U be the scheduler with a t -root routing directly to the U_D . By (8) and (16),

$$F_U = F. \quad (17)$$

For $B \in P$, let V_B be a $t|_B$ -root routing to the cells $D \subseteq B$, with children U_D . Then

$$F_{S_B} = F|_B = F_{V_B}. \quad (18)$$

Since $B \subsetneq A$, the induction hypothesis gives

$$S_B \simeq_{\text{inter}} V_B. \quad (19)$$

Substitute every V_B for S_B . This preserves the whole scheduler online: the root's reference queue is unaffected, and child B receives the same driven word—its projected pushes and one anonymous pop for every B -token. Sparse timestamps compress monotonically, so (19) applies.

The resulting scheduler has two selector levels: an outer r -selector routing to P , and an inner t -selector routing to R . Replace the outer r by t . By (13) and the colored-PIFO lemma, the entire online P -token trace is unchanged. The children therefore receive identical driven words.

Now both selector levels use exactly t . They collapse occurrence-for-occurrence. More precisely, maintain a pending set Ω of history-tagged pushes such that the two-level outer queue is Ω , labeled by P -block; U 's root queue is Ω , labeled by R -cell; the inner queue for B is $\Omega|_B$, labeled by R -cell; and corresponding U_D states are identical.

All selector keys are the same (t , arrival). If the outer queue selects $o \in B$, then o is also the minimum of $\Omega|_B$, so the inner queue selects the same occurrence, including FIFO ties. Both schedulers then call the same U_D state. A successful lower pop deletes o from all corresponding selectors; an atomic lower failure rolls both sides back identically.

Thus the two levels collapse exactly to U , and

$$S \simeq_{\text{inter}} U. \quad (20)$$

The symmetric argument, using Q, s , gives

$$T \simeq_{\text{inter}} U. \quad (21)$$

Therefore $S \simeq_{\text{inter}} T$. Restoring the universally identical **none** positions proves equality of the exact run outputs on every operation word. $\square$